Himesh EXP 10 192

1. Aim

The aim of this project is to design and develop an Inventory Management Dashboard using HTML, CSS, and Chart.js that provides a visual representation of product stock levels and inventory category distribution.

The dashboard enables users to:

- Monitor the current stock quantities of various products (Laptops, Mobiles, Headphones, Keyboards, and Monitors)
- Understand the distribution of inventory across different categories (Electronics, Accessories, and Peripherals)
- Make data-driven decisions through interactive bar and pie charts
- View a clean and responsive interface suitable for business use

2. Procedure

The following steps were followed to build the Inventory Management Dashboard:

1. Set up the HTML file (visual.html) with the basic document structure including <head> and <body> tags.
2. Included the Chart.js library via CDN (https://cdn.jsdelivr.net/npm/chart.js) inside the <head> section for chart rendering.
3. Applied CSS styling to the body, container, and canvas elements to ensure proper layout, background color (#f4f6f8), centered alignment, and responsive canvas sizes (max-height: 400px, max-width: 600px).
4. Created a <div class='container'> containing two <canvas> elements: one with id='barChart' for the bar chart, and one with id='pieChart' for the pie chart.
5. Defined sample inventory data in JavaScript:

- Products array: ['Laptops', 'Mobiles', 'Headphones', 'Keyboards', 'Monitors']
- Quantities array: [50, 120, 75, 40, 30]
- Categories array: ['Electronics', 'Accessories', 'Peripherals']
- Category counts array: [200, 100, 50]

6. Implemented a Bar Chart using Chart.js by calling new Chart() on the barChart canvas context, setting type to 'bar', supplying labels as products, and quantities as dataset data. Different background colors were assigned to each bar.
7. Implemented a Pie Chart using Chart.js on the pieChart canvas context with type set to 'pie', using categories as labels and categoryCount as data, with three distinct background colors (#36A2EB, #FF6384, #FFCE56).
8. Configured chart options for both charts including responsive: true, hidden legend for the bar chart, and visible title text ('Product Stock Levels' and 'Inventory Category Distribution').
9. Saved the file as visual.html and opened it in the browser using PowerShell command: powershell -Command "Start-Process 'c:/Users/DELL/Desktop/visual.html'"
10. Verified the output in the browser to confirm both charts rendered correctly with proper labels, colors, and data.

3. Screenshots

The following screenshots illustrate the development environment and the final rendered output of the Inventory Management Dashboard:

Screenshot 1: Dashboard Output - Bar Chart & Pie Chart

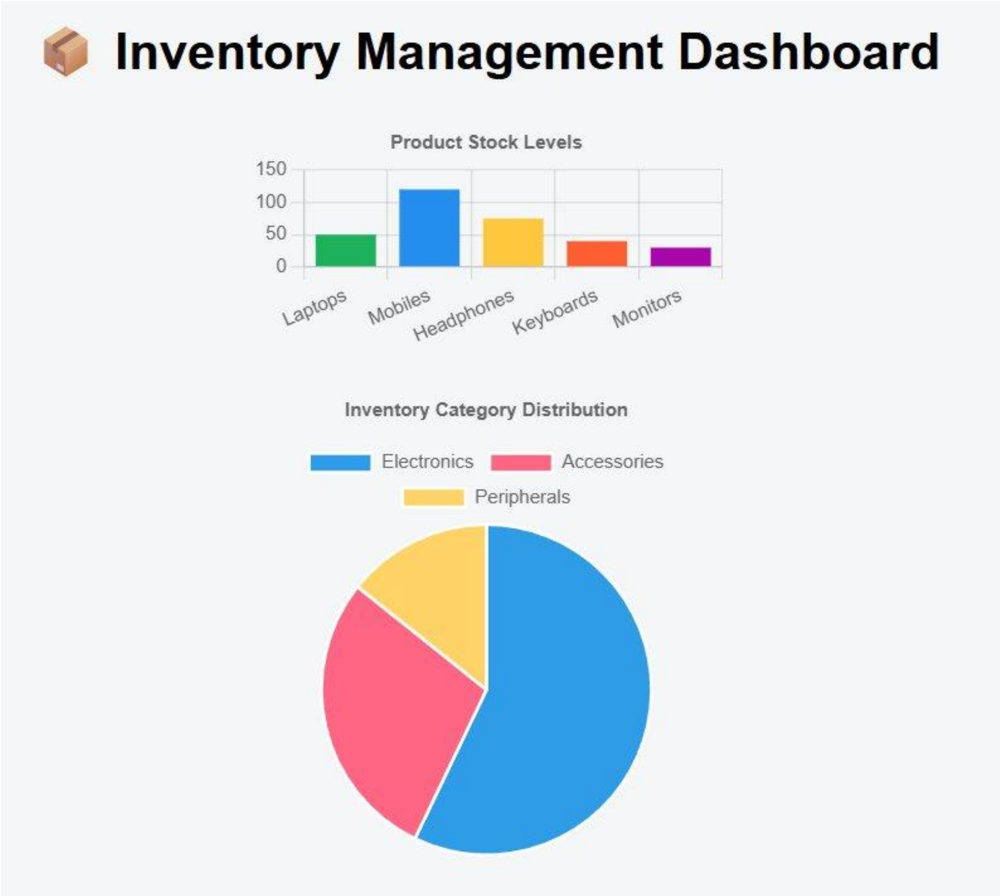

Figure 1: Rendered Inventory Management Dashboard showing Product Stock Levels (Bar Chart) and Inventory Category Distribution (Pie Chart)

Screenshot 2: HTML Code - Structure, Styles, and Bar Chart Configuration

```
visual.html ×
C: > Users > DELL > Desktop > visual.html > html > body > script
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Inventory Dashboard</title>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
    <style>
        body {
            font-family: Arial;
            background: #f4f6f8;
            text-align: center;
        }
        .container {
            width: 60%;
            max-width: 800px;
            margin: auto;
        }
        canvas {
            max-height: 400px;
            max-width: 600px;
            margin: 20px auto;
            display: block;
        }
    </style>
</head>
<body>

    <h1>📦 Inventory Management Dashboard</h1>

    <div class="container">
        <canvas id="barChart"></canvas>
        <canvas id="pieChart"></canvas>
    </div>

    <script>
        // Sample Inventory Data
        const products = ["Laptops", "Mobiles", "Headphones", "Keyboards", "Monitors"];
        const quantities = [50, 120, 75, 40, 30];

        const categories = ["Electronics", "Accessories", "Peripherals"];
        const categoryCount = [200, 100, 50];

        // 📊 Bar Chart (Product Quantities)
        const barCtx = document.getElementById('barChart').getContext('2d');
        new Chart(barCtx, {
            type: 'bar',
            data: {
                labels: products,
                datasets: [{

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS E:\vs code\html> powershell -Command "Start-Process 'c:/Users/DELL/Desktop/visual.html'"
PS E:\vs code\html>
```

Figure 2: VS Code showing the HTML structure, CSS styling, Chart.js CDN import, and Bar Chart initialization code

Screenshot 3: JavaScript Code - Bar Chart & Pie Chart Implementation

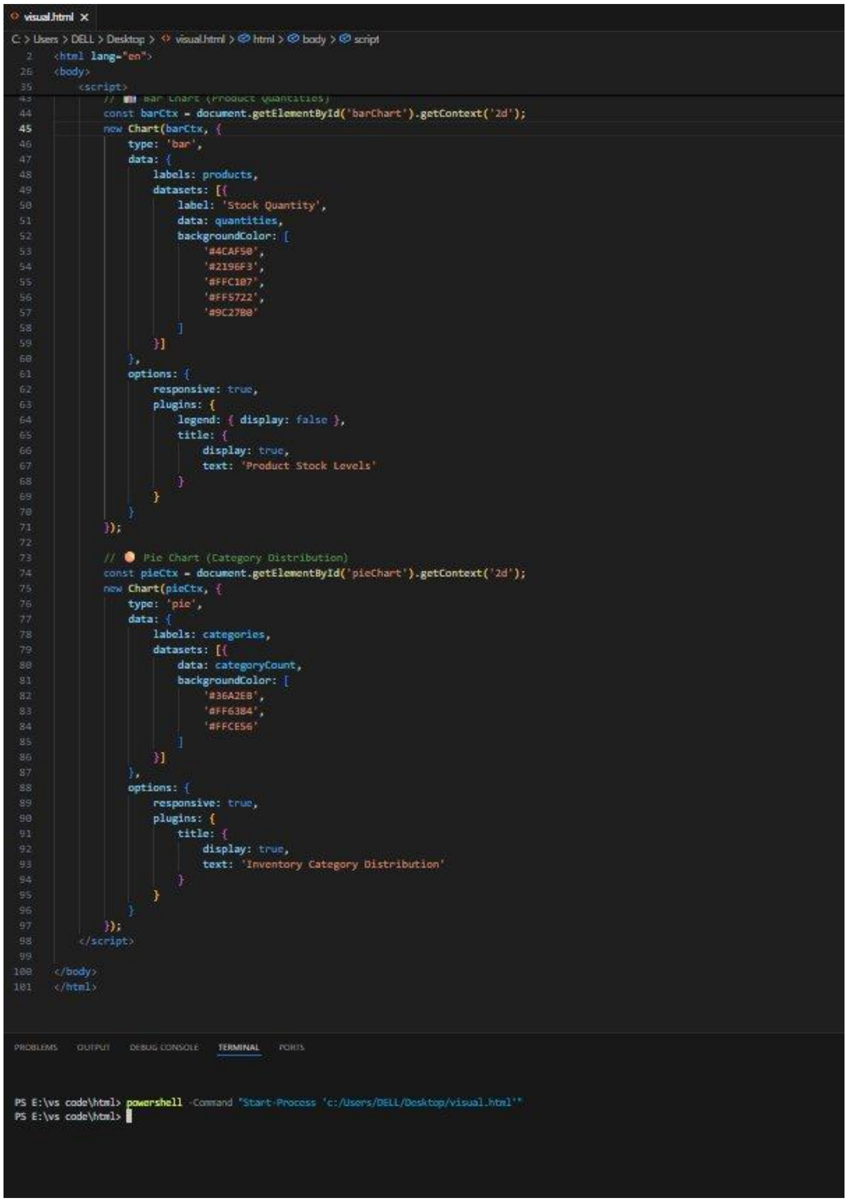

Figure 3: VS Code showing complete Chart.js configuration for both the Bar Chart (Product Quantities) and Pie Chart (Category Distribution)

4. Result

The Inventory Management Dashboard was successfully created and rendered in the browser. The following results were observed:

- The Bar Chart titled 'Product Stock Levels' correctly displayed the stock quantities for all five products. Mobiles had the highest stock (120 units), followed by Headphones (75), Laptops (50), Keyboards (40), and Monitors (30). Each bar was distinctly colored for easy identification.
- The Pie Chart titled 'Inventory Category Distribution' correctly visualized the proportion of inventory across the three categories. Electronics accounted for the largest share (200 units / ~57%), Accessories the second (100 units / ~29%), and Peripherals the smallest (50 units / ~14%).
- Both charts rendered responsively within the container div, adapting to the page width while maintaining visual clarity.
- The dashboard interface was clean, well-centered, and professional, with a light grey background (#f4f6f8) and appropriate spacing.
- The HTML file was successfully launched directly in the browser via PowerShell without any server-side setup, confirming the solution is fully client-side.

Thus, the aim of building a visual, interactive Inventory Management Dashboard using HTML and Chart.js was achieved successfully.